

Data Structure

08. 트리

8주차: Tree & BST

목차 INDEX

01 트리

- Tree
- Node & Edge
- 기본 용어들
- 재귀적 구조
- Depth & Height
- Binary Tree

02 순회

- Traversal
- Preorder
- Inorder
- Postorder
- 재귀호출 & Call Stack

03 이진 탐색 트리(BST)

- Binary Search Tree
- 탐색
- 삽입
- 삭제

01 트리

Tree

계층적 데이터의 표현 방법

📁 파일 시스템 (File System)

```
Root (내 PC)
├── Documents/
│   ├── report.pdf
│   └── notes.txt
└── Pictures/
    └── cat.jpg
```

👥 기업 조직도 (Organization Chart)

```
대표이사 (CEO)
├── 개발본부
│   ├── 백엔드팀
│   └── 프론트엔드팀
└── 마케팅본부
```

이러한 데이터를 배열이나
연결 리스트로 표현하려면?

선형/비선형 자료구조

선형 자료구조 (Linear)

종류: 배열, 연결 리스트, 스택, 큐

[A] → [B] → [C] → [D]

- 데이터가 **일직선(1:1)** 순서대로 나열됨
- 하나의 요소 뒤에는 오직 하나의 요소만 존재
- 순차적 데이터 흐름 정리에 적합

트리 구조 (Tree)

종류: 비선형 계층 자료구조 (1:N)

[A] → [B]
[A] → [C]

- 하나의 노드가 **여러 하위 노드(1:N)**를 가리킴
- 상하 **계층(Hierarchy)** 및 **분기(Branch)** 표현
- 부모-자식 관계 기반으로 표현

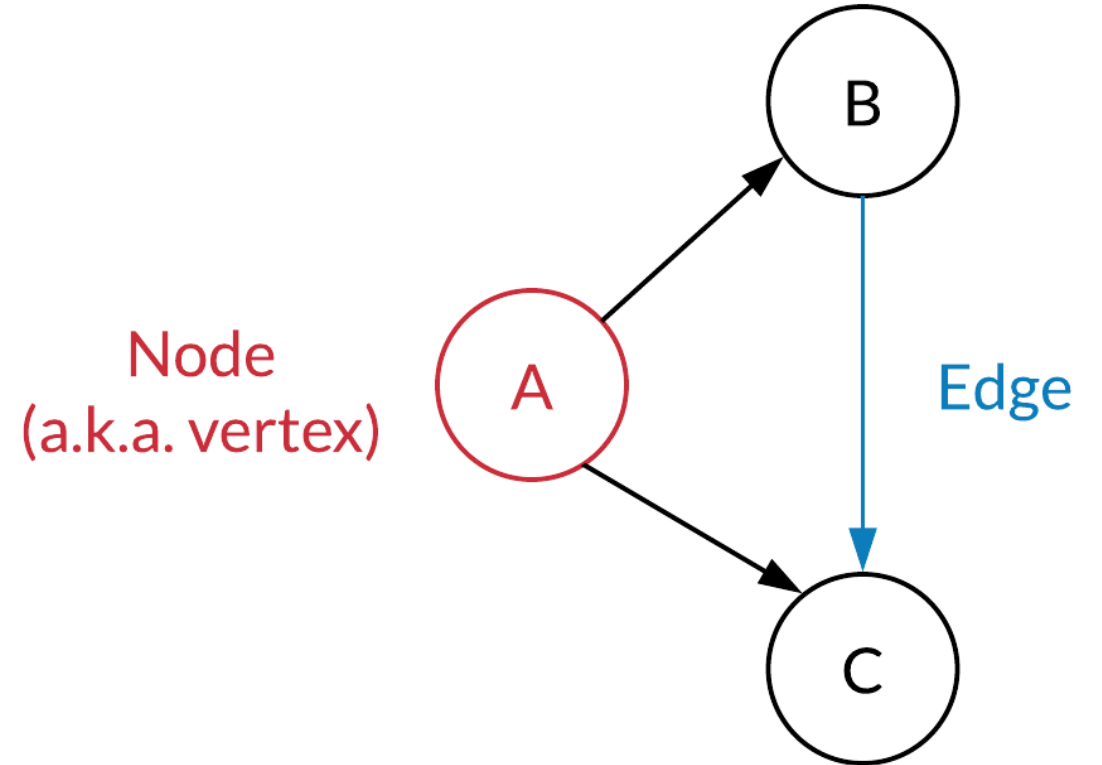
Node & Edge

● 노드 (Node / Vertex)

- 트리를 구성하는 데이터 항목 단위
- 실제 저장되는 값(Value)과 연결 정보를 가짐

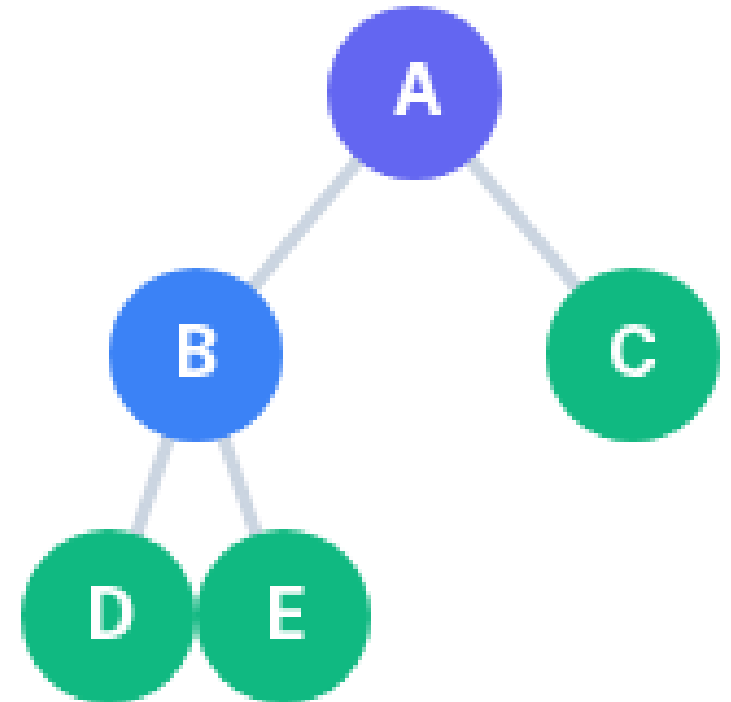
— 간선 (Edge / Link)

- 노드와 노드 사이를 연결하는 선(포인터/참조)
- 노드 간의 계층 관계를 나타냄



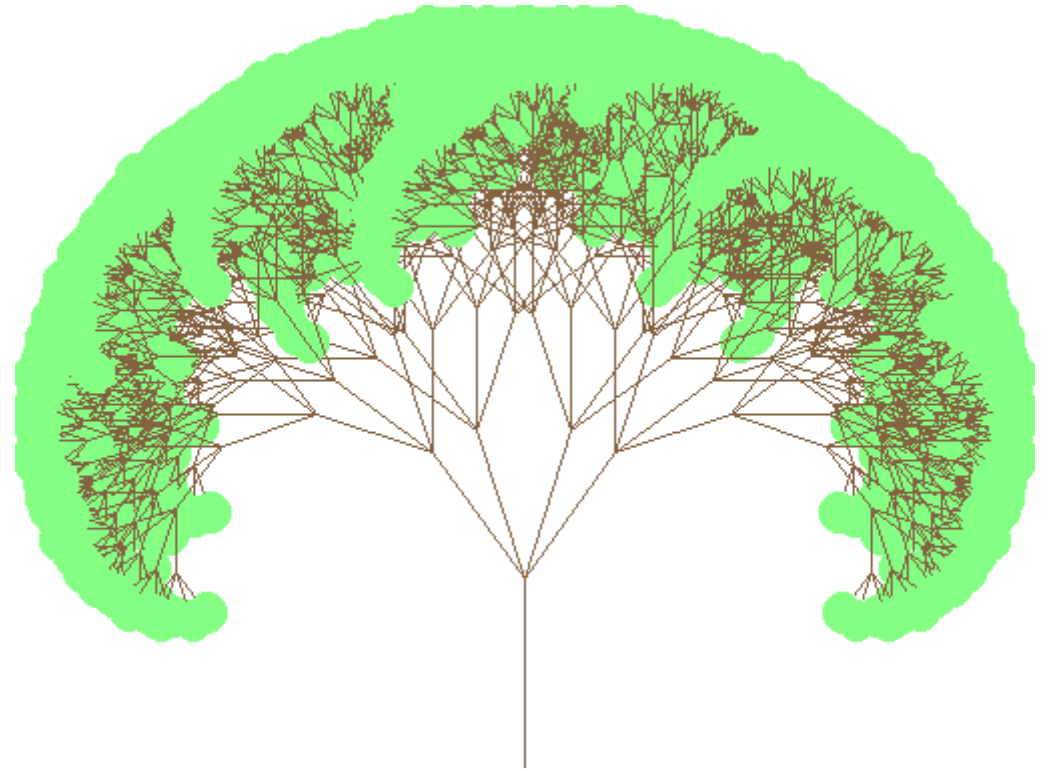
Root, Parent, Child, Leaf

용어	정의 및 의미
Root (루트)	트리의 최상위에 존재하는 단 하나의 노드 (부모가 없음)
Parent (부모)	하위 노드와 연결된 직전 상위 노드 (예: A는 B, C의 부모)
Child (자식)	상위 노드와 연결된 하위 노드 (ex) D는 B의 자식)
Leaf (단말 노드)	자식 노드가 0개인 최하단 노드 (== Terminal Node)



재귀적 구조

- 트리의 일부는 subtree라 함
- 모든 트리는 여러 subtree + root로 구성됨
- 하나의 서브트리 안에서도 여러 subtree가 있음
- 따라서, 하나의 큰 트리는 Root 하나와 하위의 작은 subtree를 가지고 있음.
그 subtree들도 내부적으로 다시 Root와 subtree들을 가짐.
(자기 자신의 구조가 계속 반복되는 구조)
- 이는 재귀를 사용하면 매우 깔끔히 반복 가능



어느 한 부분의 구조가
전체 구조와 동일한 프랙탈 구조

재귀적 구조

깊이 (Depth) — 내려가는 방향

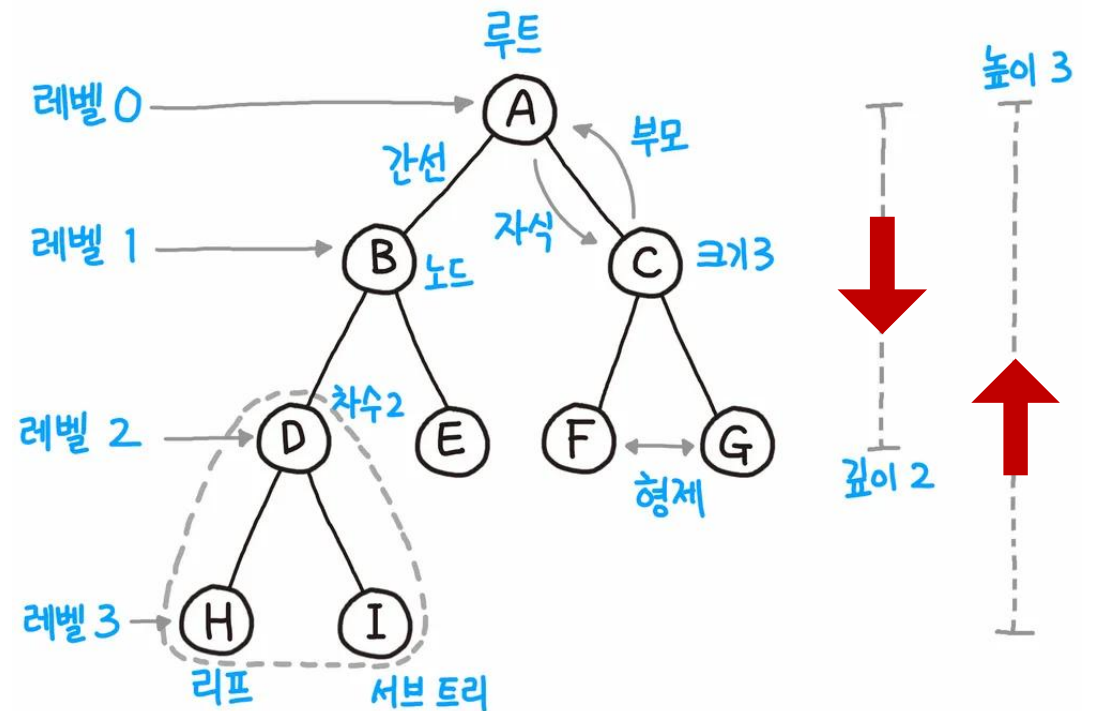
Root 노드에서 특정 노드까지 내려갈 때 거치는 간선(Edge)의 수

Root의 Depth = 0

높이 (Height) — 올라오는 방향

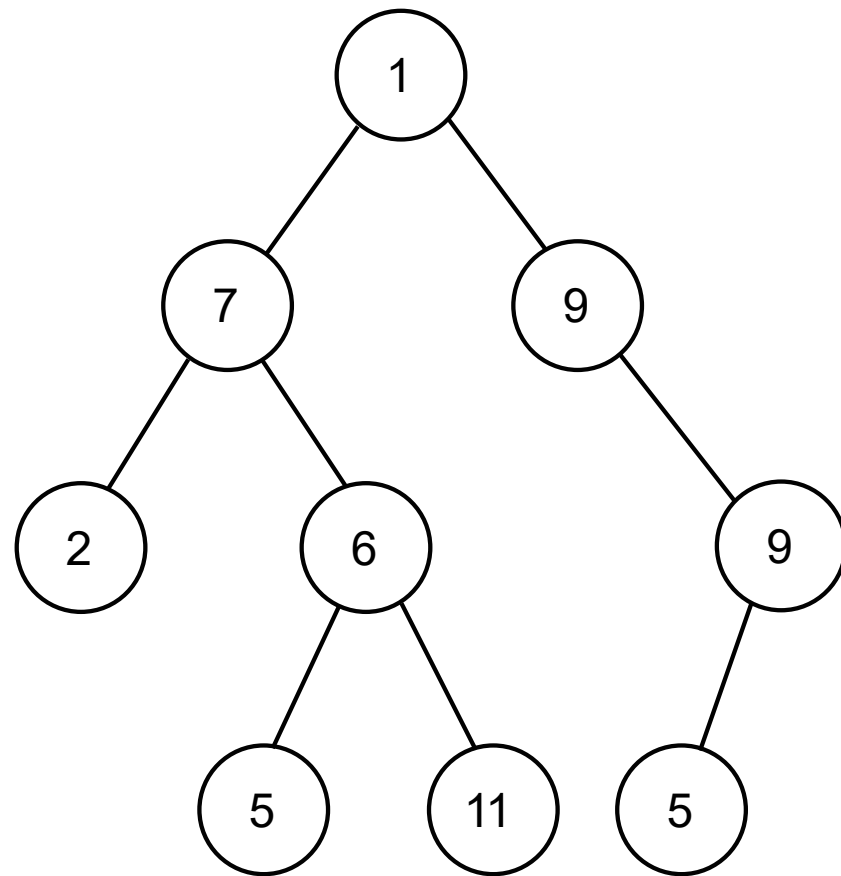
특정 노드에서 가장 먼 Leaf 노드까지 올라가는 간선(Edge)의 수

Leaf의 Height = 0 | 트리 높이 = Root 높이



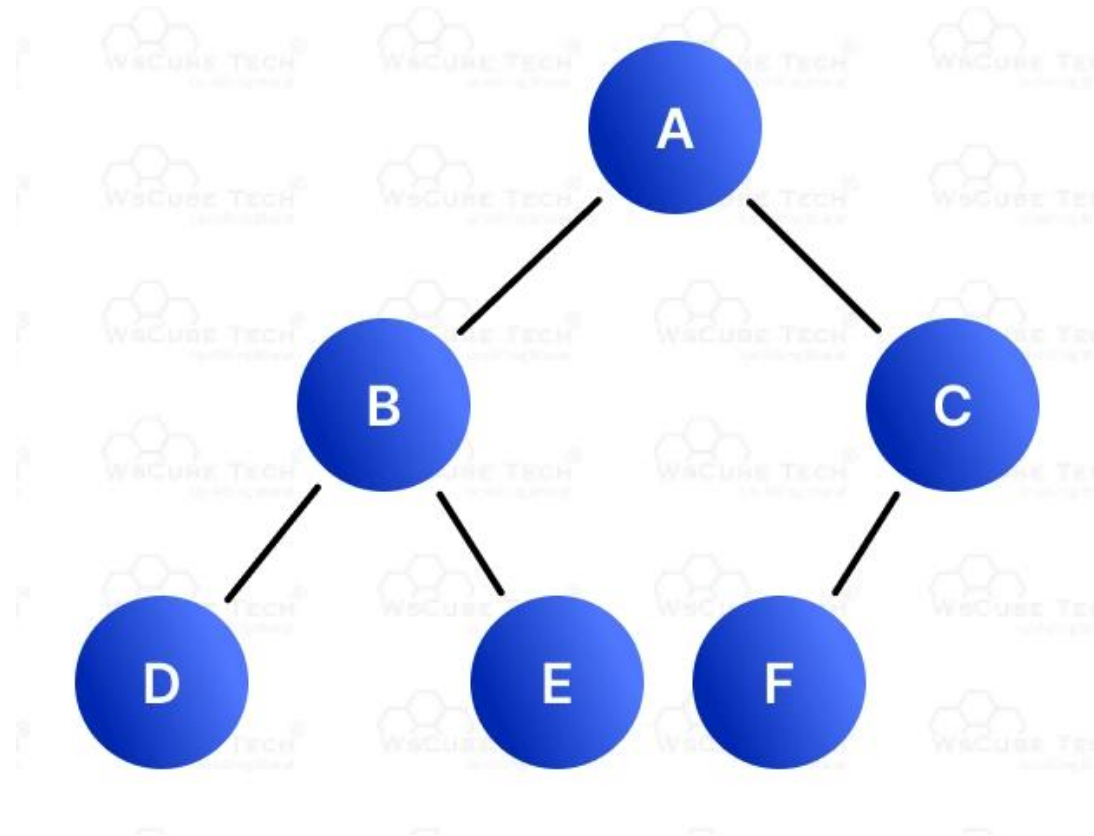
이진 트리 (Binary Tree)

- 트리의 모든 노드에 대해 자식 노드가 최대 2개 이하인 트리를 이진 트리라고 부름
- 이진 트리는 자식이 0개, 1개, 2개인 노드들로만 구성됨
- 이진 트리의 자식 노드들은 Left Child, Right Child라 부름



완전 이진 트리 (Complete Binary Tree)

1. 마지막 레벨을 제외한 모든 레벨은
노드가 가득 차 있어야 함
(왼쪽, 오른쪽 모두 노드가 있어야 함)
 2. 마지막 레벨의 노드들은 왼쪽부터 차례대로 채워져야 함
- Heap(9주차) 의 기반



02

순회

Traversal

트리의 탐색

전위 순회 (Preorder)

Root → Left → Right

부모 노드를 가장 **먼저** 방문합니다.

중위 순회 (Inorder)

Left → Root → Right

부모 노드를 **가운데(중간)**에 방문합니다.

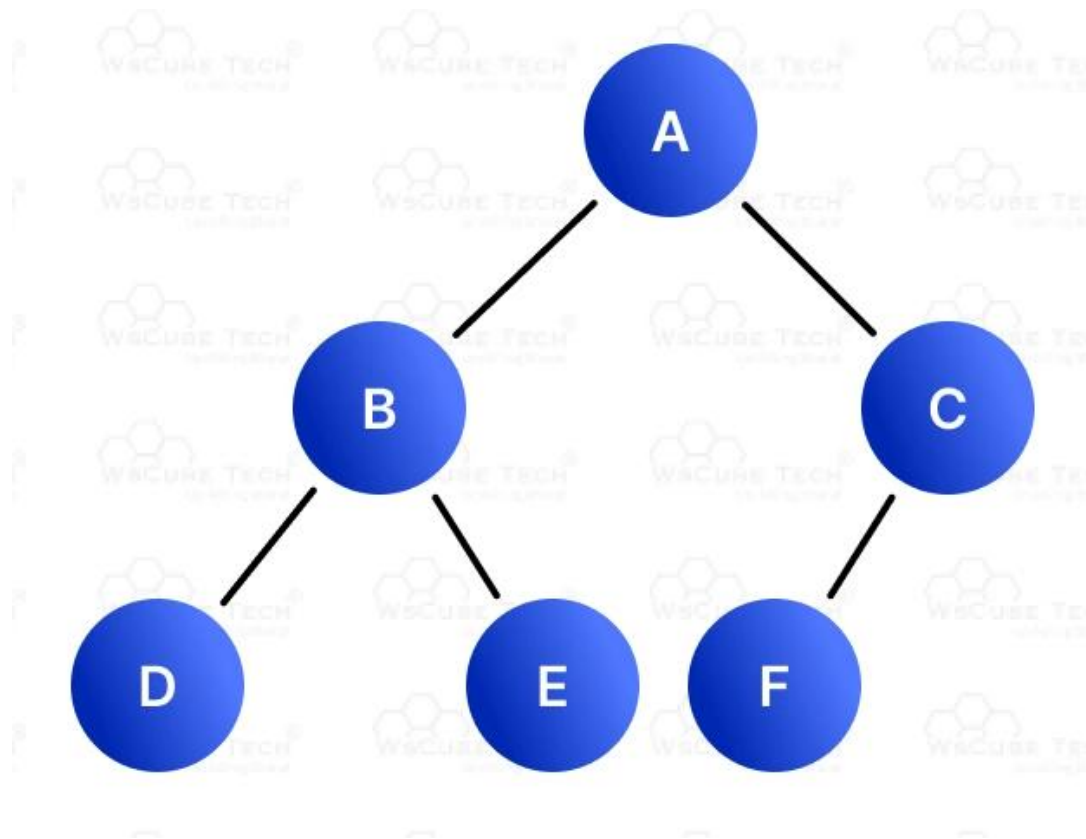
후위 순회 (Postorder)

Left → Right → Root

부모 노드를 가장 **마지막**에 방문합니다.

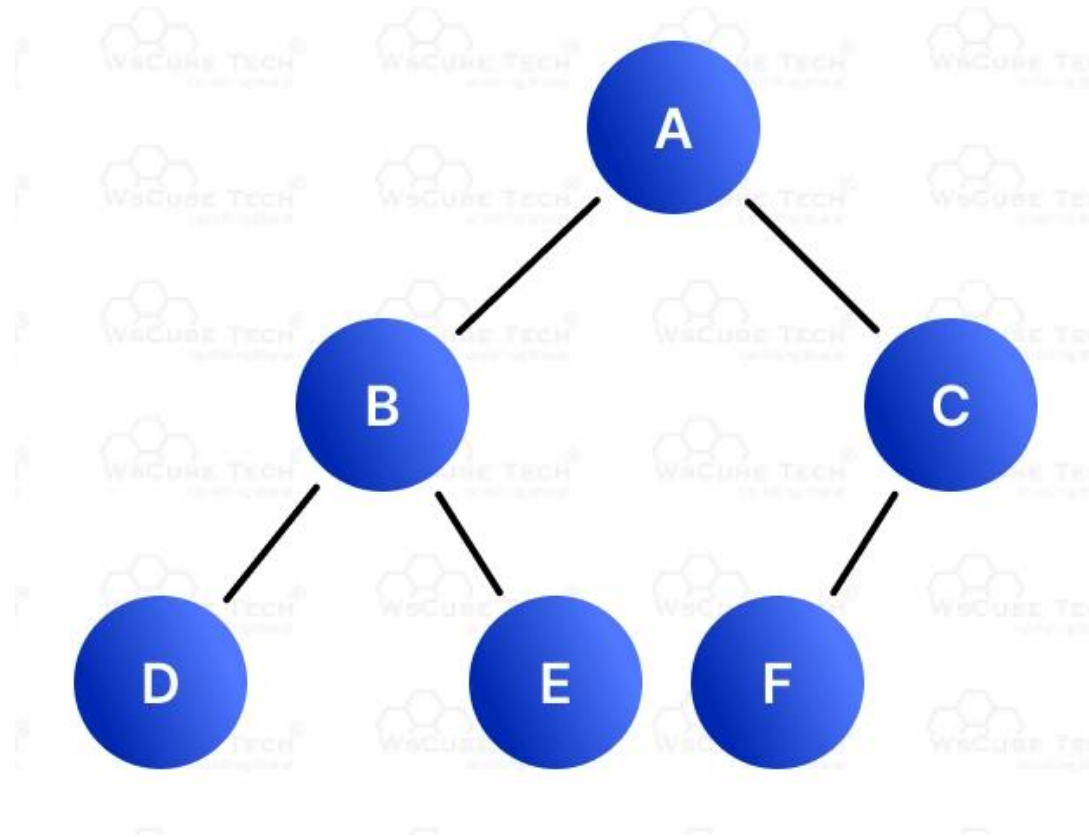
- 순회: 트리의 모든 노드를 빠짐없이 정확히 한 번씩 방문하는 과정
- Preorder: Root를 가장 먼저 방문
- Inorder: Root를 다른 노드 중간에 방문 (Left Subtree → Root → Right Subtree)
- Postorder: Root를 가장 마지막에 방문

Preorder



$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F$

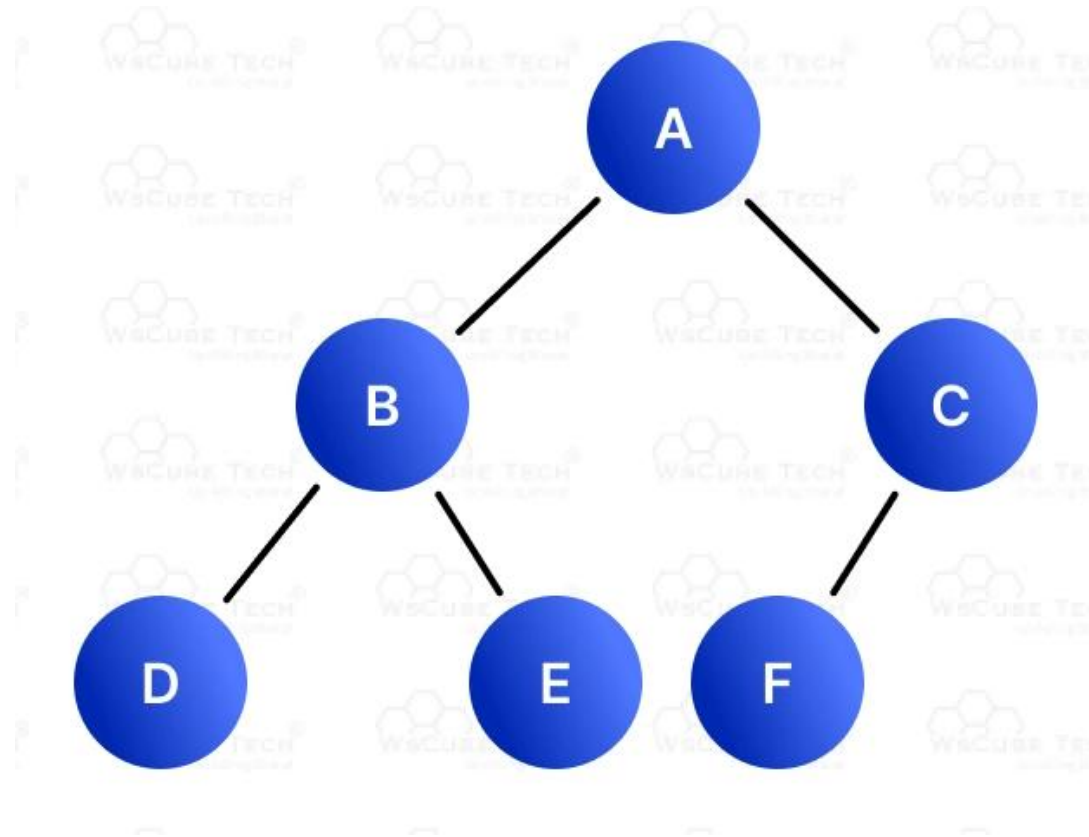
Inorder



D → B → E → A → C → F

BST에서 Inorder 하면 정렬된 결과를 얻을 수 있음

Postorder



D → E → B → F → C → A

트리 순회와 재귀

```
preorder(A)
└─▶ preorder(B)
    └─▶ preorder(D) ➔ Base Case (return)
    └─▶ preorder(E) ➔ Base Case (return)
└─▶ preorder(C)
```

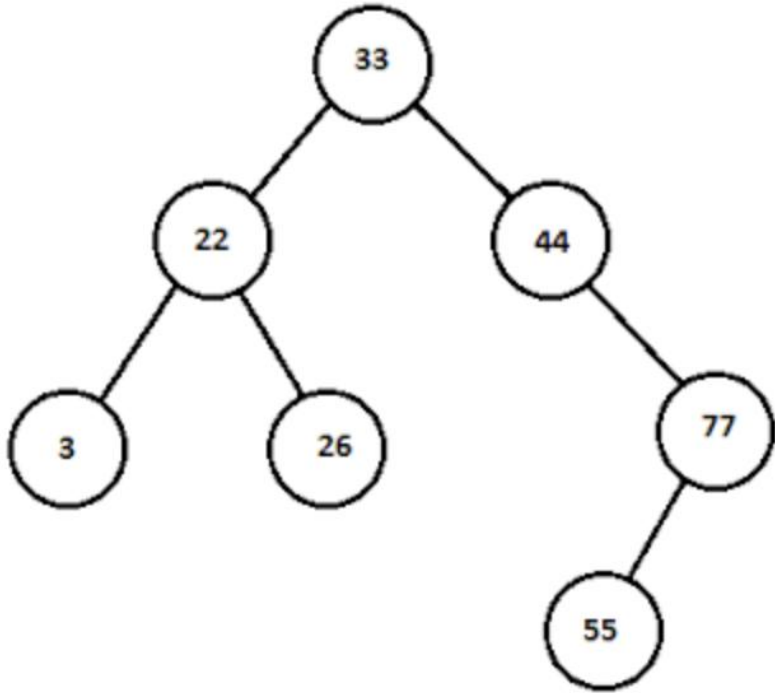
트리를 순회할 때 내부적으로는 Call Stack에
프레임이 쌓이고 되돌아오는 방식으로 구현됨
(트리를 따라 노드들을 Push 하다가 Leaf 노드에서 Pop)

03

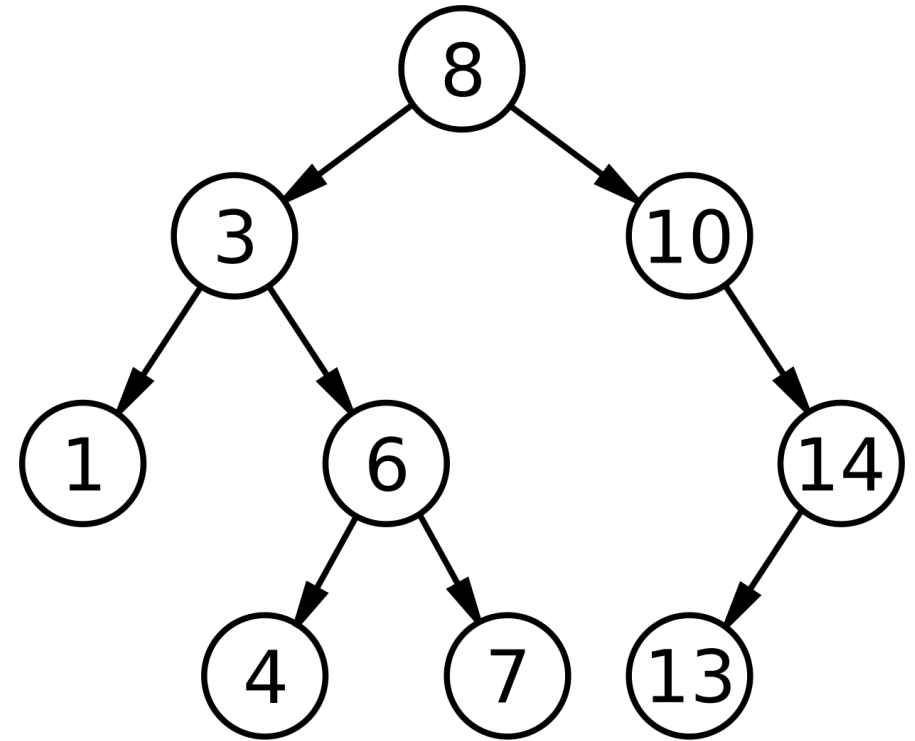
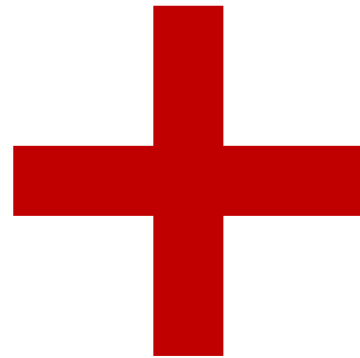
이진 탐색 트리

Binary Search Tree; BST

이진 탐색 트리



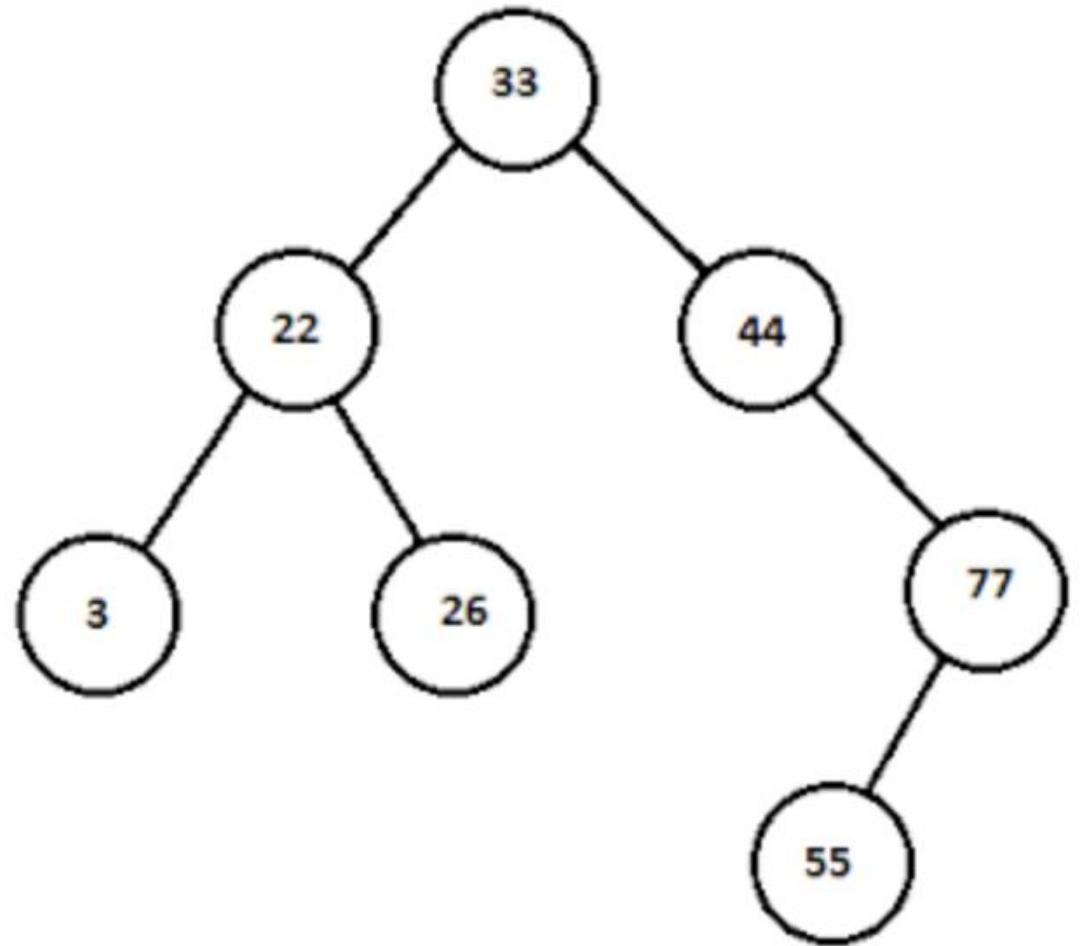
Tree



Binary
Search

이진 탐색 트리

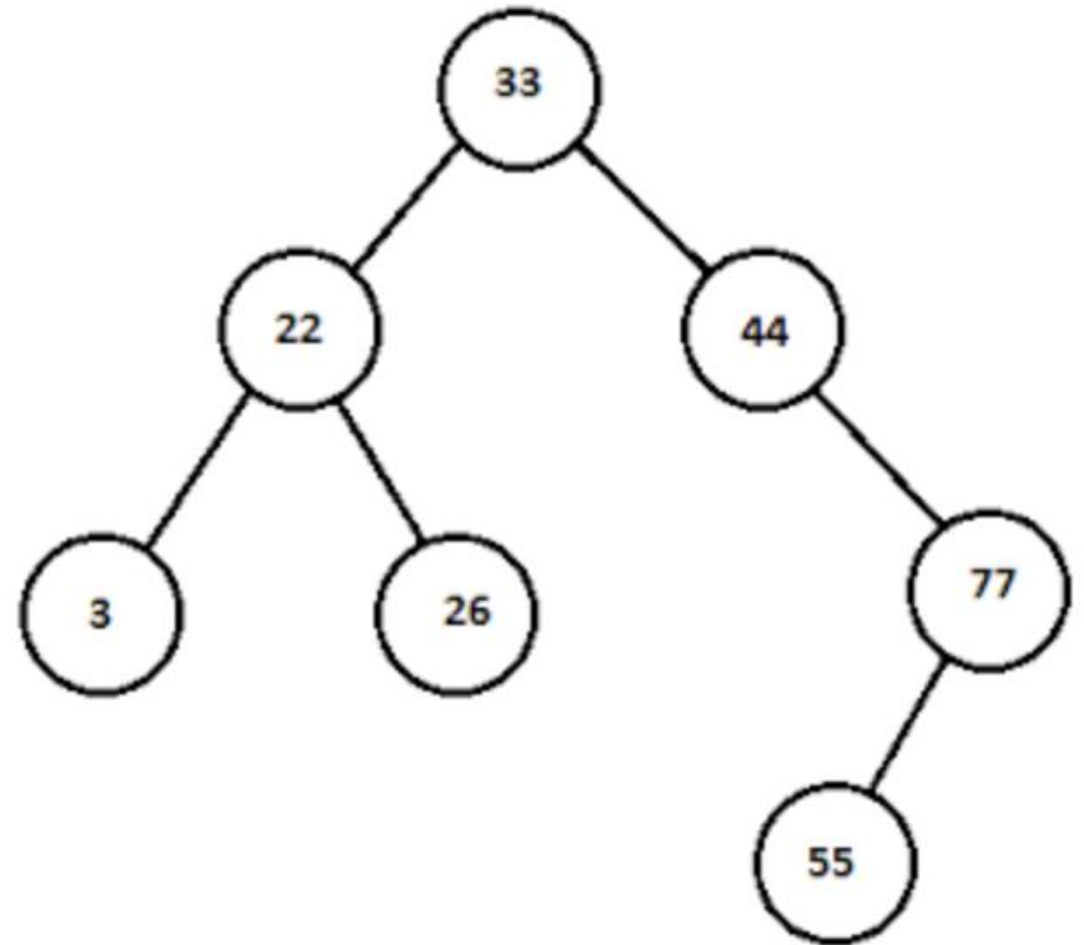
- 이진 트리 기반 구조
- 각 노드를 기준으로 작은 값은 Left Child, 큰 값은 Right Child에 배치하는 트리 구조
- Left Subtree의 모든 값 < 현재 노드의 값 < Right Subtree의 모든 값



이진 탐색 트리에서의 값 탐색

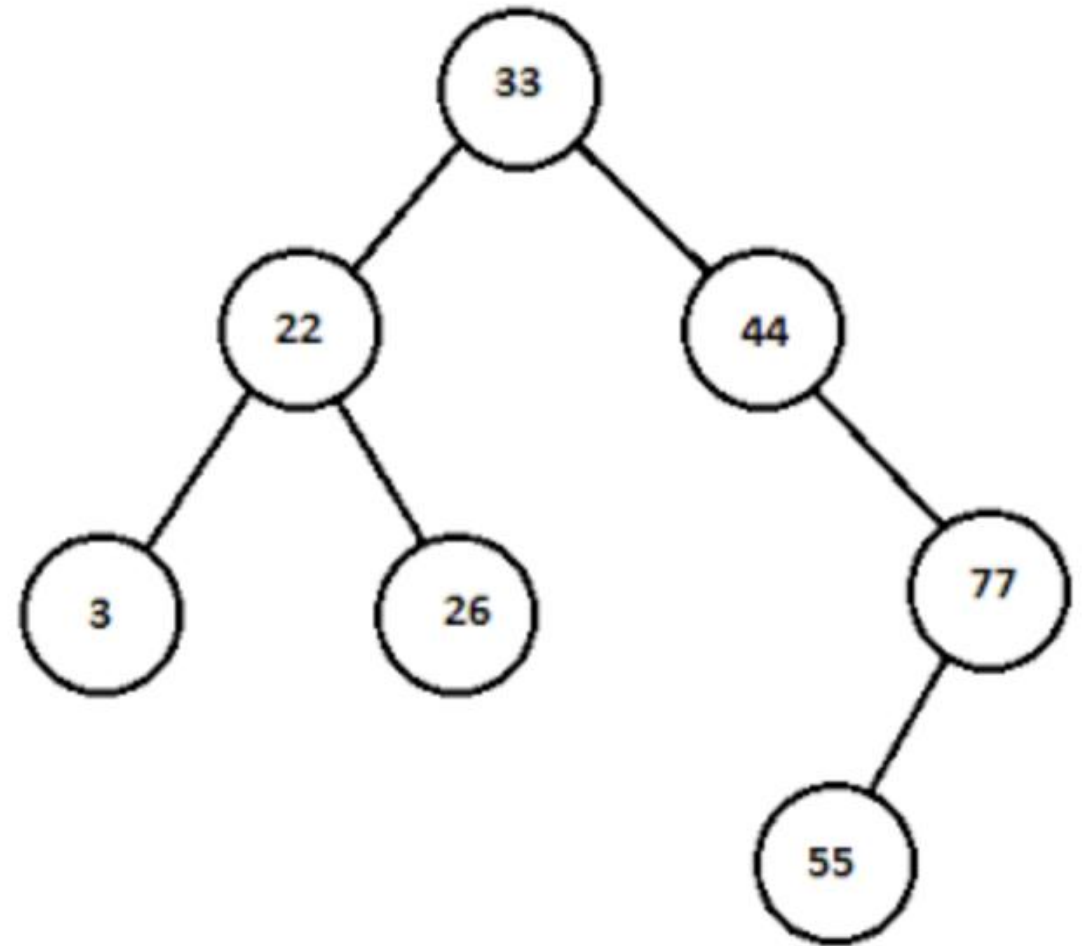
찾고 싶은 값: “26”

1. Root와 비교: 26은 33보다 작음
오른쪽 서브트리 전체 제거, 왼쪽으로 이동
2. 노드 값(22)과 비교: 26은 22보다 큼
왼쪽 서브트리 전체 제거, 오른쪽으로 이동
3. 노드 값(26)과 비교: $26 == 26$ 임
탐색 성공



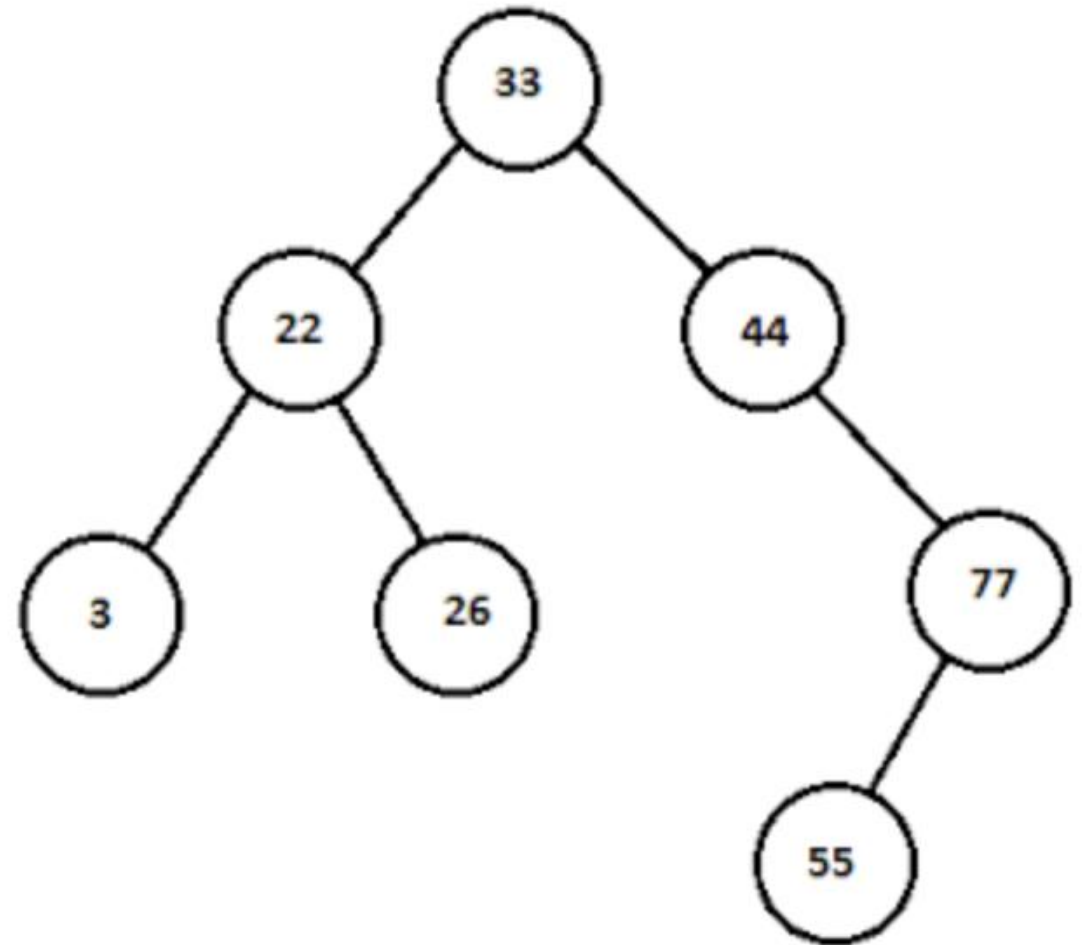
이진 탐색 트리에서의 값 삽입 넣고 싶은 값: "60"

1. Root와 비교: 60은 33보다 큼
오른쪽으로 이동
2. 노드 값(44)과 비교: 60은 44보다 큼
오른쪽으로 이동
3. 노드 값(77)과 비교: 60은 77보다 작음
왼쪽으로 이동
4. 노드 값(55)과 비교: 60은 55보다 큼
오른쪽으로 이동
5. 55번의 Right Child 자리는 비어 있는 노드임
60을 55의 Right Child로 설정



이진 탐색 트리에서의 값 삭제

- 해당 노드의 자식의 개수가 0개일 경우 (Leaf Node일 경우)
 - 해당 노드 삭제
- 해당 노드의 자식의 개수가 0개일 경우
 - 해당 노드 삭제 후 child를 부모 노드와 직접 연결



이진 탐색 트리에서의 값 삭제: 자식이 2개일 때

삭제하고 싶은 값: “22”

1. 삭제할 노드 확인: 22 노드의 자식은 3, 26으로 2개임
2. 오른쪽 서브트리의 최솟값 탐색
오른쪽 서브트리에서 왼쪽으로만 갔을 때 나오는 값은 26, 따라서 최소는 26임
3. 22 노드가 있던 자리에 26 노드 배치
4. 기존 위치의 26 노드 삭제

오른쪽 서브트리의 최솟값으로 값 대체 후
해당 최솟값 노드 삭제

